

مقایسه AVL و BST :

```
BenchmarkMain.java ×
6 ▶ public class BenchmarkMain {  MoeinH_0 *
7
8 ▶     public static void main(String[] args) {  MoeinH_0 *
9         BST<Integer,String> bst = new BST<>(Integer::compareTo);
10        AVLTree<Integer,String> avlTree = new AVLTree<>(Integer::compareTo);
11
12        long start = System.currentTimeMillis();
13        for (int i = 0; i < 100000; i++)
14            bst.insertOne(i, v: "bst");
15        bst.search(k: 50000);
16        long end = System.currentTimeMillis();
17        System.out.println("BST Search time: " + (end - start));
18
19
20        start = System.currentTimeMillis();
21        for (int i = 0; i < 100000; i++)
22            avlTree.insertOne(i, v: "bst");
23        avlTree.search(k: 50000);
24        end = System.currentTimeMillis();
25        System.out.println("AVLTree Search time: " + (end - start));
26    }
```

```
Run  Benchmark ×
C:\Users\KASRA\.jdk\openjdk-24.0.1\bin\java.exe "-javaagent:C:\Program Files\Je
BST Search time: 11390
AVLTree Search time: 28
```

داده های 0 تا 100000 را اینزرت میکنیم و سپس کلید 50000 را سرچ میکنیم همانطور که مشخص است درخت AVL حدود 400 برابر سریع تر است و این به دلیل این است که در درخت BST سرچ در $O(n)$ انجام میشود اما در درخت AVL در $O(\log(n))$.

مقایسه : Scan, AVL, Hash

```
Benchmark2Main.java x HashIndex.java commands.txt
8 public class Benchmark2Main { new *
9
10 public static void main(String[] args) { new *
11     ArrayCollection arrayCollection = new ArrayCollection();
12     AVLTree<Integer, Student> avlTree = new AVLTree<>(Integer::compareTo);
13     HashIndex<Integer, Student> hashIndex = new HashIndex<>(size: 1000);
14
15     for (int i = 0; i < 10000000; i++){
16         Student s = new Student(i, name: "moein", gpa: i % 21);
17         arrayCollection.insertOne(s);
18         avlTree.insertOne(i, s);
19         hashIndex.insertOne(i, s);
20     }
21
22     long start = System.currentTimeMillis();
23     arrayCollection.findById(80000000);
24     long end = System.currentTimeMillis();
25     System.out.println("ArrayCollection Search time (O(n)): " + (end - start));
26
27     start = System.currentTimeMillis();
28     avlTree.search(k: 80000000);
```

Run Benchmark2 x

```
C:\Users\KASRA\.jdk\openjdk-24.0.1\bin\java.exe "-javaagent:C:\Program Files\JetBrains
ArrayCollection Search time (O(n)): 82
AVLTree Search time (O(log(n))): 0
HashIndex Search time (O(1)): 2
```

همانطور که مشخص است اسکن خیلی بیشتر از دو ایندکس دیگر زمان لازم دارد برای سرچ اما دو ایندکس دیگر به دلیل کم بودن تعداد رکورد تفاوت انچانی ندارند به دلیل ارور مموری امکان ثبت رکورد بیش ازین مقدار نبود.

تحلیل InvertexIndex :

توضیح دهید چرا برای پیدا کردن کلمه "Ali" در نامها، استفاده از Index Inverted بسیار سریعتر از search substring روی تک تک رکوردهاست؟

به دلیل اینکه اگر از InvertedIndex استفاده کنیم نیازی نداریم تک تک رکورد ها رو چک کنیم و در واقع هر کلمه تبدیل به یک ایندکس میشود که با $O(1)$ میتوان به آن عبارت دسترسی پیدا کرد.